

CS3212 Algorithms

Aya Zirikly
09/10/2026

Stirling's Approximation & Final Proof

Stirling's Formula for Factorials: $n! \approx \sqrt{2 * \pi * n} * (n / e)^n$

$$\log_2(n!) = \log_2((n / e)^n) + \log_2(\sqrt{2 * \pi * n})$$

$$\log_2((n / e)^n) = n * \log_2(n / e) = n \log_2 n - n \log_2 e$$

$$\log_2(\sqrt{2 * \pi * n}) = (1/2) \log_2(2 * \pi * n) = (1/2) \log_2(2 * \pi) + (1/2) \log_2 n$$

$$\log_2(n!) = n \log_2 n - n \log_2 e + O(\log n)$$

Stirling's Approximation & Final Proof

When analyzing growth as n approaches infinity, we look at which term grows the fastest:

- $n \log_2 n$ grows extremely fast.
- $n \log_2 e$ (approx $1.44n$) grows linearly.
- $O(\log n)$ grows very slowly.

$$\log_2(n!) = \Theta(n \log n)$$

Algorithm	Best-Case Time	Average-Case Time	Worst-Case Time	Auxiliary Space	Stable?	Paradigm
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	Yes	Divide & Conquer
QuickSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$	No	Divide & Conquer
HeapSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$	No	Selection / Heap
QuickSelect	$\Theta(n)$	$\Theta(n)$	$\Theta(n^2)$	$O(\log n)$	No	Prune & Conquer
Median-of-Medians	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$O(\log n)$	No	Divide & Conquer

Module 3: Heaps, HeapSort & Non-Comparison Sorting

Divide-and-conquer trees → heap structures

Topics Covered:

- ❑ Implicit Binary Trees & Heap Representation
- ❑ Max-Heap & Min-Heap Properties
- ❑ Heap Operations (MAX-HEAPIFY, BUILD-MAX-HEAP)
- ❑ The HeapSort Algorithm (In-Place $O(n \log n)$)
- ❑ Priority Queue Abstract Data Type
- ❑ Breaking $\Omega(n \log n)$: Counting Sort, Radix Sort, Bucket Sort

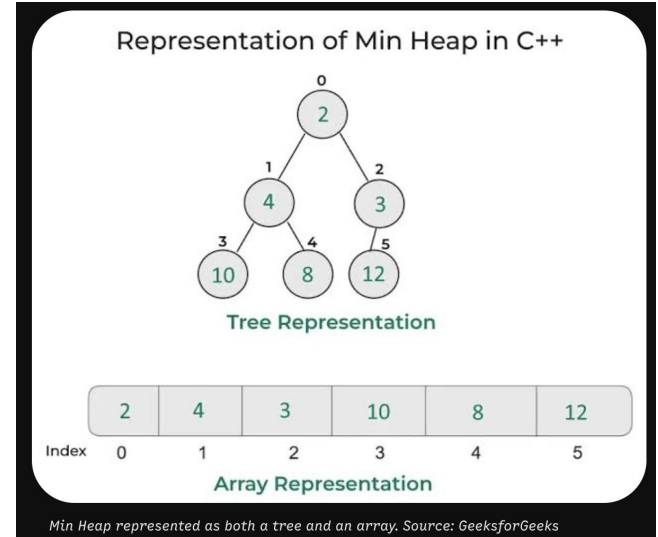
Why heaps are useful

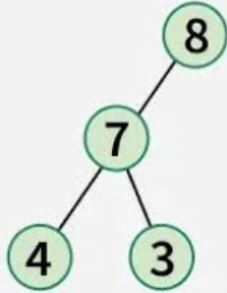
- ❑ Get the max (or min) element immediately in **$O(1)$** time.
- ❑ Fast **$O(\log n)$** insertions without re-sorting the whole dataset.
- ❑ Finds the top k items in **$O(n \log k)$** time instead of fully sorting everything.
- ❑ Works directly inside a flat array using **$O(1)$** extra memory, avoiding recursion stack limits.

What is a heap?

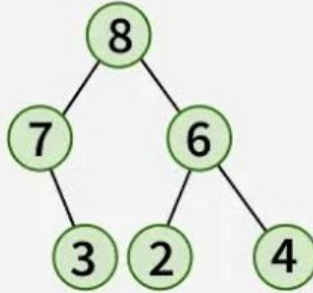
A **heap** is a specialized tree-based data structure that satisfies the following properties

- ❖ **Complete Binary Tree:** Filled top-to-bottom, left-to-right with no gaps in levels.
- ❖ **Heap Order Property:** Every parent is larger than its children (Max-Heap) or smaller (Min-Heap).
- ❖ **Array-Backed:** Stored as a flat, single-dimensional array without pointers.
- ❖ **Fast Extrema Access:** Root node always holds the absolute maximum or minimum element in **$O(1)$** time.
- ❖ **Logarithmic Operations:** Insertion and deletion run in **$O(\log n)$** time to restore ordering.

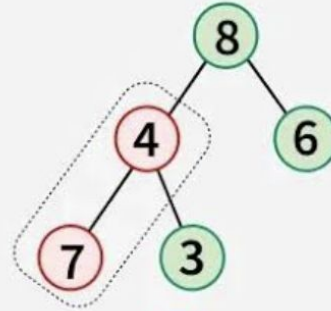




Not a Complete Binary Tree



Not a Complete Binary Tree



Violates Max Heap Property

Invalid Max Heaps

Heap structural constraints and ordering rules. Source: *GeeksforGeeks*

The Max-Heap and Min-Heap Properties

Max-Heap properties:

- ❖ For every node i other than root: $A[\text{PARENT}(i)] \geq A[i]$.
- ❖ The largest element in the subtree is at the root $A[1]$.
- ❖ Primary applications: HeapSort and Max-Priority Queues.

The Max-Heap and Min-Heap Properties

Min-Heap properties:

- ❖ For every node i other than root: $A[\text{PARENT}(i)] \leq A[i]$.
- ❖ The smallest element in the subtree is root $A[1]$.
- ❖ Primary applications: Min-Priority Queues (e.g., Dijkstra's shortest path, Huffman coding).

Structural Properties & Height Bounds

Heap Height Definition:

- ❖ Height of node i : Number of edges on longest simple downward path to a leaf.
- ❖ Height of heap: Height of tree = $\text{floor}(\log_2 n)$.

Height Bounds

Height is defined as the number of edges on the longest path from the root to a leaf.

A tree with 1 node ($n = 1$) $\rightarrow$ height 0: $\log_2(1) = 0 \rightarrow \text{floor}(0) = 0$

A tree with 2 nodes ($n = 2$) $\rightarrow$ height 1: $\log_2(2) = 1 \rightarrow \text{floor}(1) = 1$

A tree with 3 nodes ($n = 3$) ?

Height Bounds

Height is defined as the number of edges on the longest path from the root to a leaf.

A tree with 1 node ($n = 1$) $\rightarrow$ height 0: $\log_2(1) = 0 \rightarrow \text{floor}(0) = 0$

A tree with 2 nodes ($n = 2$) $\rightarrow$ height 1: $\log_2(2) = 1 \rightarrow \text{floor}(1) = 1$

A tree with 3 nodes ($n = 3$) $\rightarrow$ height 1: $\log_2(3) \sim 1.58 \rightarrow \text{floor}(1.58) = 1$

Structural Properties & Height Bounds

Number of Nodes (n)	$\log_2(n)$	$\text{floor}(\log_2 n)$	Tree Height (h)
n = 1	0	0	0
n = 2 to 3	1 to 1.58	1	1
n = 4 to 7	2 to 2.80	2	2
n = 8 to 15	3 to 3.90	3	3

Array Tree Navigation Arithmetic

Parent-Child Navigation (1-Based Indexing)

- ❖ $\text{PARENT}(i): \text{floor}(i / 2)$
- ❖ $\text{LEFT}(i): 2 * i$
- ❖ $\text{RIGHT}(i): 2 * i + 1$

Core Navigation Formulas

In a 1-based indexed array, parent and child positions are calculated directly from any node index i using standard arithmetic:

PARENT NODE

$$\text{PARENT}(i) = \lfloor i / 2 \rfloor$$

Integer division by 2 finds the parent node.

LEFT CHILD

$$\text{LEFT}(i) = 2 \times i$$

Multiplying index by 2 yields the left child.

RIGHT CHILD

$$\text{RIGHT}(i) = (2 \times i) + 1$$

Multiplying index by 2 plus 1 yields the right child.

Why 1-Based Indexing?

- **Zero Offset:** Index 0 remains unallocated, aligning pure multiples of 2 with tree levels.
- **Clean Math:** No extra +1 or -1 shifts required compared to 0-based indexing.
- **Textbook Standard:** Used across primary computer science algorithms literature (e.g., CLRS).

Visual Mapping Example

Level-Order Tree to Array

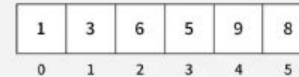
Elements map sequentially level-by-level from top to bottom, left to right.

Index	0	1	2	3	4	5
Value	-	50	30	20	15	10

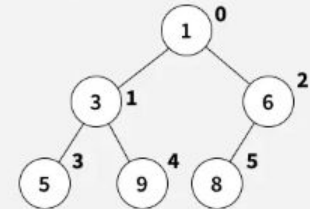
Root resides at index 1. Subtrees align seamlessly into adjacent array slots.

A heap is often represented as an array rather than a tree structure. In this array-based representation, for any element at index i , the left child is at index $2i + 1$ and the right child is at index $2i + 2$. The parent of any element at index i is at index $(i-1)/2$.

actual representation



pictorial representation



Representation of a Binary Heap

Worked Example: Node at Index $i = 3$

Target Element: $A[3] = 20$

Applying the navigation formulas to index $i = 3$:

PARENT(3): $\lfloor 3 / 2 \rfloor = 1 \rightarrow A[1] = 50$

LEFT(3): $2 \times 3 = 6 \rightarrow A[6] = 8$

RIGHT(3): $(2 \times 3) + 1 = 7 \rightarrow A[7] = 16$

Array State Trace

Idx	1 (P)	2	3 (i)	4	5	6 (L)	7 (R)
Val	50	30	20	15	10	8	16

The formulas instantly compute exact target indices in $O(1)$ time without pointer traversals.

Hardware Efficiency: Bitwise Shifts



Left Shift (LEFT)

```
i << 1
```

Shifting bits left by 1 multiplies by 2 instantly in a single CPU clock cycle.



Right Shift (PARENT)

```
i >> 1
```

Shifting bits right by 1 divides by 2 with automatic floor rounding.



Shift + Bitwise OR (RIGHT)

```
(i << 1) | 1
```

Multiplying by 2 and setting the lowest bit computes the right child seamlessly.

Key Takeaways

- ✓ **PARENT(i):** $\text{floor}(i / 2)$ maps upward to parent.
- ✓ **LEFT(i):** $2 * i$ maps downward to left child.
- ✓ **RIGHT(i):** $2 * i + 1$ maps downward to right child.
- ✓ **Performance:** Executes in 1 CPU cycle via single bit-shift operations.

Array Tree Navigation Arithmetic

Parent-Child Navigation (0-Based Indexing):

- $\text{PARENT}(i): \text{floor}((i - 1) / 2)$
- $\text{LEFT}(i): 2 * i + 1$
- $\text{RIGHT}(i): 2 * i + 2$

Core Navigation Formulas

In standard 0-indexed arrays, parent and child node positions are calculated from node index i using these offset equations:

PARENT NODE

$$\text{PARENT}(i) = \lfloor (i - 1) / 2 \rfloor$$

Subtract 1 and divide by 2 (floored).

LEFT CHILD

$$\text{LEFT}(i) = (2 \times i) + 1$$

Multiply index by 2 and add 1.

RIGHT CHILD

$$\text{RIGHT}(i) = (2 \times i) + 2$$

Multiply index by 2 and add 2.

Why 0-Based Indexing?

- **Language Standard:** Matches native array allocation in C, C++, Java, Python, and JavaScript.
- **Zero Memory Waste:** Slot $A[0]$ holds the root node directly without leaving index 0 unused.
- **Offset Convenience:** Eliminates custom wrapper boundaries or 1-based artificial array shifts.

Visual Mapping Example

0-Indexed Tree to Array

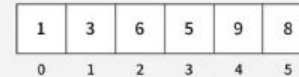
The root node sits at index 0, followed by level-order traversal elements sequentially.

Index	0	1	2	3	4
Value	50	30	20	15	10

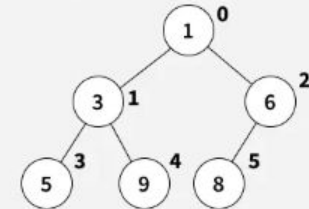
Root resides at index 0. Level-by-level elements map continuously into memory.

A heap is often represented as an array rather than a tree structure. In this array-based representation, for any element at index i , the left child is at index $2i + 1$ and the right child is at index $2i + 2$. The parent of any element at index i is at index $(i-1)/2$.

actual representation



pictorial representation



Representation of a Binary Heap

Worked Example: Node at Index $i = 2$

Target Element: $A[2] = 20$

Applying 0-based formulas to index $i = 2$:

$$\text{PARENT}(2): \lfloor (2 - 1) / 2 \rfloor = 0 \rightarrow A[0] = 50$$

$$\text{LEFT}(2): (2 \times 2) + 1 = 5 \rightarrow A[5] = 8$$

$$\text{RIGHT}(2): (2 \times 2) + 2 = 6 \rightarrow A[6] = 16$$

Array State Trace

Idx	0 (P)	1	2 (i)	3	4	5 (L)	6 (R)
Val	50	30	20	15	10	8	16

Computes exact tree neighbor indices in constant $O(1)$ time using native zero-indexed array access.

Hardware Efficiency: Bitwise Shifts



Left Child (LEFT)

```
 $(i \ll 1) | 1$ 
```

Shift bits left by 1 and set the lowest bit to 1 (calculates $2*i + 1$ instantly).



Parent Node (PARENT)

```
 $(i - 1) \gg 1$ 
```

Subtract 1 and shift right by 1 bit to divide by 2 with automatic integer flooring.



Right Child (RIGHT)

```
 $(i + 1) \ll 1$ 
```

Add 1 then shift left by 1 bit to compute $(i + 1) * 2$ (equivalent to $2*i + 2$).

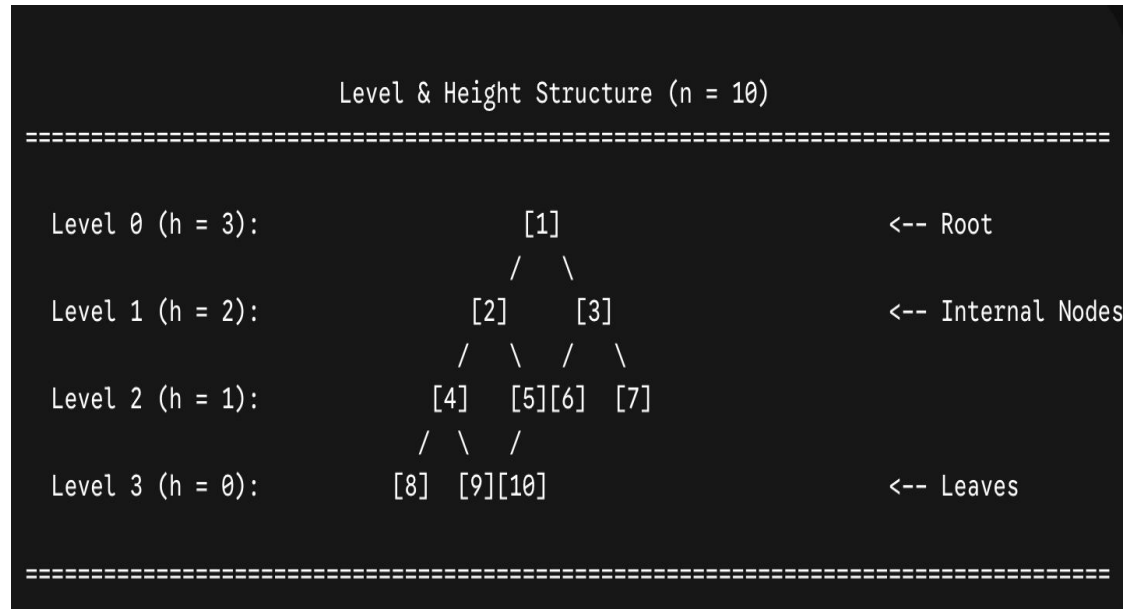
Key Takeaways

- ✓ **PARENT(i):** $\text{floor}((i - 1) / 2)$ or $(i - 1) \gg 1$
- ✓ **LEFT(i):** $2 * i + 1$ or $(i \ll 1) | 1$
- ✓ **RIGHT(i):** $2 * i + 2$ or $(i + 1) \ll 1$
- ✓ **Zero Overhead:** Integrates natively with standard array indexing across modern programming languages.

Structural Properties & Height Bounds

Key Structural Bounds for Heap of Size n :

- Total tree height is $h = \text{floor}(\log_2 n)$.
- Leaf nodes reside at array indices $\text{floor}(n / 2) + 1$ to n
- Total leaf node count is $\text{ceil}(n / 2)$.
- Total internal (non-leaf) node count is $\text{floor}(n / 2)$.
- At most $\text{ceil}(n / 2^{(h + 1)})$ nodes exist at height h .



- ❑ Total tree height is $h = \text{floor}(\log_2 n)$.
- ❑ Leaf nodes reside at array indices $\text{floor}(n / 2) + 1$ to n
- ❑ Total leaf node count is $\text{ceil}(n / 2)$.
- ❑ Total internal (non-leaf) node count is $\text{floor}(n / 2)$.
- ❑ At most $\text{ceil}(n / 2^{(h + 1)})$ nodes exist at height h .

At most $\text{ceil}(n / 2^{(h + 1)})$ nodes exist at height h

Define Height 0 (The Base Case) Height $h = 0$ corresponds to the **leaf nodes** at the very bottom of the tree.

- Substituting $h = 0$ into $2^{(h + 1)}$ gives $2^{(0 + 1)} = 2^1 = 2$.
- Formula yields: $\text{ceil}(n / 2)$.
- **Meaning:** Exactly half the nodes in a complete binary tree (rounded up) are leaves.

Step 2: Move Up the Tree (Halving at Each Level) Every node at height $h + 1$ is a parent to nodes at height h . Because every parent in a binary tree has up to 2 children, moving up 1 height level cuts the maximum possible number of nodes **in half**:

- Nodes at height $h \leq (\text{Nodes at height } 0) / (2^h)$

Step 3: Combine the Terms Substitute the leaf count ($\text{ceil}(n / 2)$) into the halving equation:

- Nodes at height $h \leq \text{ceil}(n / 2) / (2^h) = \text{ceil}(n / (2 * 2^h)) = \text{ceil}(n / 2^{(h + 1)})$

Representing Binary Trees Efficiently in Arrays

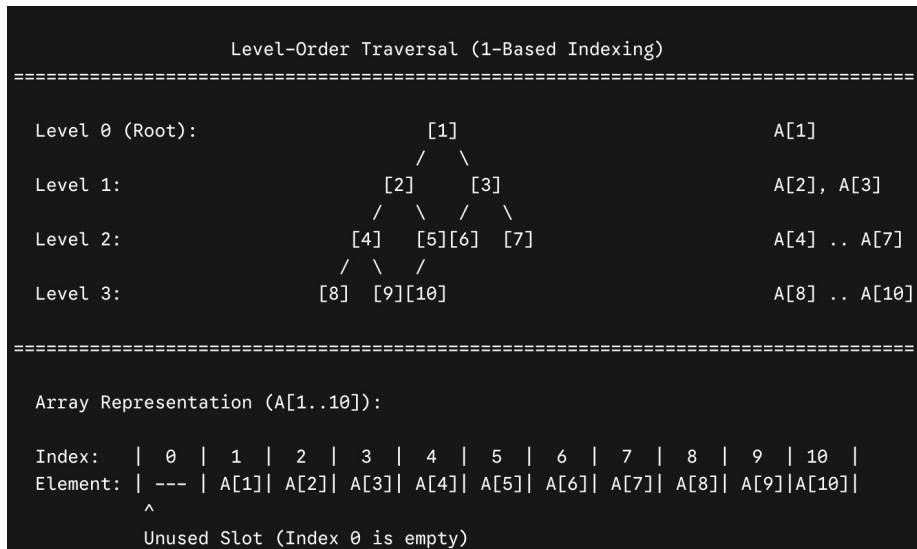
Store a nearly complete binary tree as a contiguous array without explicit node pointers.

Array Mapping Mechanics:

- Root node is stored at index 1 (1-based indexing).
- Nodes are mapped level-by-level from top to bottom, left to right.

Mapping a binary tree level-by-level (also called **level-order traversal**): read each row of the tree from top to bottom, and within each row, moving strictly left to right into adjacent array indices.

Representing Binary Trees Efficiently in Arrays



- Root node is stored at index 1 (1-based indexing).
- Nodes are mapped level-by-level from top to bottom, left to right.

Mapping a binary tree level-by-level (also called **level-order traversal**): read each row of the tree from top to bottom, and within each row, moving strictly left to right into adjacent array indices.

Representing Binary Trees Efficiently in Arrays

Store a nearly complete binary tree as a contiguous array without explicit node pointers.

Array Mapping Mechanics:

- ❖ Root node is stored at index 1 (1-based indexing).
- ❖ Nodes are mapped level-by-level from top to bottom, left to right.

Two Critical Array Attributes:

- ❖ `length[A]`: physical memory capacity of the array (the maximum number of elements it can hold).
- ❖ `heap-size[A]`: Number of active heap elements stored in array A ($\text{heap-size}[A] \leq \text{length}[A]$).